



Generation AI

Nurturing Future Innovators

«ԱԲ սերունդ» ծրագիր
10-րդ դասարան

«ՀԱՄԱԿԱՐԳՉԱՅԻՆ ԳԻՏՈՒԹՅՈՒՆ»

PYTHON

ԾՐԱԳՐԱԿՈՐՄԱՆ ԼԵՃՎՈՎ

Խնդիրների ժողովածու

Տարբերակ 1.0

Մաս 2

Նախաբան

Ժամանակակից աշխարհում ծրագրավորումը միայն հմտություն չէ, այլ կարևոր գործիք: Python-ը ներկայումս ամենապարզ և բազմակողմանի ծրագրավորման լեզուներից մեկն է: Իր պարզ կառուցվածքի, ընթերցելիության և լայն կիրառության շնորհիվ այն հիանալի հնարավորություն է տալիս միջին և ավագ դասարանների աշակերտներին առաջին քայլերը կատարել ծրագրավորման ոլորտում: Բացի այդ, արհեստական բանականության ոլորտը գրեթե ամբողջությամբ հիմնված է Python-ի և նրա հզոր գրադարանների վրա, ինչպիսիք են NumPy-ն և Matplotlib-ը, որոնք օգտագործում են տվյալների վերլուծության, մոդելավորման և վիզուալացման համար:

Python-ի ընդգրկումն ուսումնական ծրագրերում, հատկապես «ԱԲ սերունդ» նախագծի շրջանակում, ցույց է տալիս աճող պահանջարկը ինչպես միջին, այնպես էլ ավագ դպրոցում: Խնդրագիրքը ստեղծվել է այս պահանջարկը բավարարելու և ուսումնական գործընթացը համապատասխան նյութերով ապահովելու համար՝ ներառելով ծրագրավորման գործնական խնդիրների կիրառություններ:

Խնդրագիրքն ուղղված է սովորողների կողմավորման հմտությունների զարգացմանը՝ տրամադրելով բարդ խնդիրների լուծման մեթոդաբանություն: Այն հնարավորություն է տալիս յուրաքանչյուր սովորողին աշխատել իր գիտելիքների մակարդակին համապատասխան առաջադրանքների վրա: Առաջադրանքները կառուցված են այնպես, որ խրախուլվածմանս են ուսումնասիրել իրական կյանքի կիրառական խնդիրներ, նպաստեն գործնական աշխատանքի և ինքնուրույն ուսումնառության զարգացմանը: Ժողովածուի այս մասում կենտրոնացել ենք ավելի կոմպլեքս խնդիրների լուծմանը, որոնք ուսանողներին կմղեն խորացնել իրենց գիտելիքները և կիրառել դրանք Նոր իրավիճակներում:

Python լեզվով «Խնդիրների Ժողովածու» ձեռնարկի մշակող-փորձագետ՝

Միհրան Ավագյան, ԳՏՃՄ մանկավարժ-փորձագետ

Python լեզվով «Խնդիրների Ժողովածու» ձեռնարկի վերանայող-փորձագետ՝

Ավագ Սայան, Տեխնոկրթության փորձագետ

Երախտիքի խոսքեր.

Մենք մեր երախտագիտությունն ենք հայտնում «ԱԲ սերունդ» ծրագրի շուրջ ձևավորված մասնագիտական համայնքին՝ մշակվող նյութերը պարբերաբար վերանայելու և բարելավմանն ուղղված դիտարկումներ տրամադրելու համար: Խնդրագրքի այս տարբերակը գտնվում է փորձարկման փուլում:

Հեղինակային իրավունքը պատկանում է Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամին:

Նախաբան.....	2
Թեմա 1. Ֆունկցիաներ.....	4
Թեմա 2. Ֆայլեր (Նիշք).....	7
Թեմա 3. Գրադարաններ (libs).....	8
NumPy.....	8
Matplotlib Pyplot.....	13
Թեմա 4. Խորացված ֆունկցիաներ (Advanced functions).....	25
Թեմա 4. Ռեկուրսիա (Recursion).....	32
Օգտագործված գրականություն և արտաքին հղումներ.....	35

Թեմա 1. Ֆունկցիաներ

1. Կազմել ֆունկցիա, որ տպում է " Hello world":
2. Կազմել ֆունկցիա, որ ստանում է անուն, որպես արգումենտ և տպում է ողջունի խոսք:
3. Կազմել ֆունկցիա, որը ստանում է ծննդյան տարեթիվը և վերադարձնում է տարիքը:
4. Կազմել ֆունկցիա, որը ստանում է գինու արտադրության տարեթիվը և ընթացիկ տարին: Ֆունկցիան ստուգում է, արդյոք գինին պիտանի է, եթե այն պիտանի է մինչև 10 տարի:
5. Կազմել ֆունկցիա, որ ստանում է երկու թիվ, տպում է նրանց գումարը:
6. Կազմել ֆունկցիա, որը ստանում է թվեր և վերադարձնում է նրանց քանակը: Ֆունկցիան պետք է ընդունի անսահմանափակ արգումենտներ:
7. Կազմել ֆունկցիա, որը որպես արգումենտ ընդունում է թվերից կազմված ցուցակ և վերադարձնում է բոլոր թվերի գումարը:
8. Կազմել ֆունկցիա, որը ստանում է շրջանագծի շառավիղը և տպում է նրա մակերեսը և երկարությունը:
9. Կազմել ֆունկցիա, որը ստանում է անհրաժեշտ ներկի քանակը լիտրով, հաշվում է թե քանի տարրա պետք է գնել, և որքան ներկ կավելանա: Յուրաքանչյուր տարր պարունակում է 5 լ ներկ:
10. Կազմել ֆունկցիա, որը ստանում է 2 ամբողջ թվիվ և տպում է նրանց միջև եղած բոլոր ամբողջ թվերը:
11. Կազմել ֆունկցիա, որը ստանում է 2 թիվ և վերադարձնում է նրանց ամենամեծ ընդհանուր բաժանարարը:
12. Կազմել ֆունկցիա, որը ընդունում է անուն, տարի, և լռելայն հոբի, և վերադարձնում է տեղեկությունները:
13. Կազմել ֆունկցիա, որը ողջունում է բոլոր այն մարդկանց, ում անունները տրամադրված են kwargs-ի մեջ:
14. Կազմել ֆունկցիա, որը ստանում է թիվ, վերադարձնում է true, եթե թիվը պարզ է և false հակառակ դեպքում: Կազմել երկրորդ ֆունկցիան, որը ստանում է թիվ և տպում է մինչև այդ թիվը եղած բոլոր պարզ թվերը: Երկրորդ ֆունկցիան պետք է օգտագործի առաջին ֆունկցիան:
15. Ռոբոտը գտնվում է (0, 0) կետում և շարվում է **right**, **forward**, **left**, **backward** հրամաններով, ամեն հրամանի դեպքում գնալով մեկ միավոր առաջ, հետ, ձախ կամ աջ: Գրել ֆունկցիա, որը կստանա հրամանների հաջորդականությունը որպես ցուցակ և կտալի վերջնական դիրքի կոորդինատը:
16. Գրեք ֆունկցիա, որը ստանում է ուսանողների անունների և նրանց գնահատականների ցանկը բառարանով, և վերադարձնում է, թե ովքեր են անցել, իսկ ովքեր չեն անցել: Անցողիկ են համարվում 60 և ավել միավոր հավաքած ուսանողները:

Օրինակ:

Մուտք

Ելք

{"Անի": 70, "Գոռ": 55, "Եվա": 85} ("Անցել է", ["Անի", "Եվա"]), ("Չի անցել", ["Գոռ"])

Կազմել ֆունկցիա, որը ստանում է հետևյալ տեսքի բառարան`

17. Կազմել **ֆունկցիա**, որը ստանում է հետևյալ տեսքի բառարան՝

Python

```
{ "10 լումա": a, "20 լումա": b, "50 լումա ": c, "1 դրամ": d, "3 դրամ": e,
  "5 դրամ": f, "10 դրամ": g
}
```

որտեղ *a*, *b*, *c*, *d*, *e*, *f*, *g* թվերը ներկայացնում են յուրաքանչյուր տեսակի մետաղադրամի քանակը: **Ֆունկցիան** վերադարձնում է այդ մետաղադրամների ընդհանուր գանգվածը:

Մետաղադրամների գանգվածներն են՝

- 10 լումա - 0.59 գրամ
- 20 լումա - 0.75 գրամ
- 50 լումա - 0.93 գրամ
- 1 դրամ - 1.39 գրամ
- 3 դրամ - 1.63 գրամ
- 5 դրամ - 1.98 գրամ
- 10 դրամ - 2.3 գրամ

Դասակարգման ալգորիթմներ

18. Գրել **ֆունկցիա**, որը ստանում է թվերից կազմված ցուցակ և այն դասակարգում է աղպջակային դասակարգման ալգորիթմով (Bubble Sort):

Այս ալգորիթմը համեմատում է հարևան տարրերը և փոխում նրանց տեղերը, եթե նրանք սխալ կարգով են: Կրկնվում է մինչև ամբողջ ցանկը լինի սորտավորված:

19. Գրել **ֆունկցիա**, որը ստանում է թվերից կազմված ցուցակ և այն դասակարգում է ընտրության դասակարգման ալգորիթմով (Selection Sort):

Այս ալգորիթմը գտնում է ամենափոքր տարրը և տեղադրում այն ցանկի սկզբում, ապա նույնը անում է չդիտարկելով ցուցակի առաջին տարրը:

20. Կազմել **ֆունկցիա**, որը բառարանի տեսքով ստանում է օրվա ընթացքում օգտագործվող սնունդի անունները և կալորիականությունը հետևյալ տեսքով՝

Python

```
{'Cake': 800, 'Pizza': 500, 'Coke': 139}
```

Ֆունկցիան վերադարձնում է օրվա ընթացքում օգտագործված ընդհանուր կալորիաների թիվը:

21. Կազմել ֆունկցիա, որը որպես արգումենտ ստանում է ֆիլմերի անունների և դրանց տևողությունները բառարանով օրինակ՝ {'Interstellar': 169, 'Inception': 148, 'Dune': 155}: Տպում է այն ֆիլմերի անունները, որոնց տևողությունը չի գերազանցում 150 րոպեն:
22. Կազմել ֆունկցիա, որը ստանում է կայանատեղի մուտքի և ելքի ժամանակները տողով (օրինակ՝ '10:30' և '13:45'): հաշվարկում է կայանման տևողությունը և վերադարձնի վճարելի գումարը (1 ժամ = 300 դրամ): Ժամանակը կլորացվում է ըստ ժամերի , այսինքն 2 ժամ 15 րոպեն հաշվարկվում է 3 ժամ:
23. Կազմել ֆունկցիա, որը ստանում է երկու արգումենտ: Առաջին արգումենտը բառարան է, որը ներկայացնում է քաղաքը այդ քաղաք այցելելու համար անհրաժեշտ գումարը՝ (օրինակ {'Dilijan': 30000, 'Kapan': 50000, 'Sevan': 20000}), երկրորդ արգումենտը ամբողջ թիվ է, որը ցույց է տալիս առկա գումարը: Ֆունկցիան տպում է այն քաղաքների անունները, որոնք հնարավոր է այցելել այդ գումարով:

Օրինակ

Մուտք

40.0000 {'Dilijan': 30000, 'Kapan': 50000, 'Sevan': 20000}

Ելք

'Dilijan' , 'Sevan'

24. Կազմել ֆունկցիա, որը ստանում է երկու բառարան, որպես արգումենտ: Առաջինը ներկայացնում է վաճառված սովորական և ՎԻՊ տոմսերի քանակը՝ {'Սովորական': 10, 'ՎԻՊ': 5}: Երկրորդը տոմսերի գները {'Սովորական': 3000, 'ՎԻՊ': 5000}: Ֆունկցիան հաշվում և վերադարձնում է ընդհանուր վաճառքի գումարը:
25. Կազմել ֆունկցիա, որը ստանում է անվանական աշխատավարձի չափը և աշխատակցի ծննդյան տարեթիվը և վերադարձնում է մաքուր աշխատավարձը: Անվանական աշխատավարձից պահվում է 20% եկամտահարկ, 5% սոց վճար (եթե աշխատակիցը ծնվել է 1974թ-ից հետո, հակառակ դեպքում սոց վճար չի պահվում), և դրոշմանիշային վճար ըստ հետևյալ աղյուսակի՝

Աշխատավարձի չափ, դրամ	Դրոշմանիշային վճար, դրամ
Մինչև 100.000	1500
100,001 – 200,000	3000
200,001 – 500,000	5500
500,001-ից ավել	8500

Թեմա 2. Ֆայլեր (Նիշք)

1. Գրել ծրագիր, որը ստանում է տեքստ օգտատիրոջից և պահում այն ֆայլում *data.txt*:
2. Գրել ծրագիր, որը բացում է *data.txt* ֆայլը և տպում դրա պարունակությունը (*data.txt* ֆայլը ինքնուրույն ստեղծեք համակարգչում):
3. Գրել ծրագիր, որը բացում է *data.txt* ֆայլը և տպում դրա պարունակությունը տող առ տող:
4. Գրել ծրագիր, որը բացում է *data.txt* ֆայլը և տպում դրա պարունակությունը բառ առ բառ: Բառերը իրարից առանձնացված են բացատով:
5. Գրել ծրագիր, որը բացում է *data.txt* ֆայլը և հաշվում է ֆայլում բոլոր բառերի քանակը:
6. Գրել ծրագիր, որը բացում է *data.txt* ֆայլը և տպում այն տողերը, որոնց երկարությունը փոքր է 10-ից:
7. Գրել ծրագիր, որը բացում է *data.txt* ֆայլը և հաշվում է, թե քանի տող կա այնտեղ:
8. Գրել ծրագիր, որը բացում է *data.txt* ֆայլը և փոխարինում է ֆայլում հանդիպող "Python"-ն բառը "Programming" բառով: Եթե ֆայլում չկա Python բառը ոչինչ չի փոխում:
9. Գրել ծրագիր, որը միավորում է համակարգչում գտնվող *data1.txt* և *data2.txt* ֆայլերի պարունակությունը և պահում այն նոր *merged.txt* ֆայլում:
10. Գրել ծրագիր, որը կկարդա CSV ֆայլը և կտպի ֆայլի պարունակությունը տող առ տող:
11. Գրել ծրագիր, որը ստեղծում է *data.txt* ֆայլի կրկնօրինակը հետևյալ անունով *copy_data.txt*:
12. Գրել ծրագիր, որը կարդում է *data.txt* ֆայլի պարունակությունը և տպում է այն տողերը, որոնցում կան "Error" բառը:
13. Գրել ծրագիր, որը գտնում է *data.txt* ֆայլի ամենաերկար և ամենակարճ տողերը՝ տպելով դրանց բովանդակությունն ու երկարությունը:
14. Գրել ծրագիր, որը հաշվում է *data.txt* ֆայլում 'A' և 'a' տառերի ընդհանուր քանակը:
15. Գրել ծրագիր, որը հաշվում է *data.txt* ֆայլում դատարկ տողերի քանակը:
16. Գրել ծրագիր, որը հաշվում է *data.txt* ֆայլի տողերի երկարությունների միջինը:
17. *digit.txt* ֆայլում կան միայն թվեր, և յուրաքանչյուր թիվ գրված է առանձին տողի վրա: Գրել ծրագիր, որը կբացի ֆայլը, կկարդա թվերը և կհաշվի դրանց գումարը:
18. Գրել ծրագիր, որը *data.txt* ֆայլի բոլոր ձայնավորների ` A , E, I, O, U, a, e, i, o, u -ի փոխարեն կգրի # : Օգտագործեք տողերի *replace* մեթոդը:
19. Գրել ծրագիր, որը բացում է *data.txt* ֆայլը և տպում միայն այն տողերը, որոնք պարունակում են ավելի քան 5 բառ:
20. Գրել ծրագիր, որը կբացի *data.txt* ֆայլը և հաշվի բոլոր թվանշանների միջինը:
21. 21-ից 24 խնդիրները լուծելիս օգտվեք *os* գրադարանից:
22. Գրել ծրագիր, որը կջնջի *data.txt* ֆայլը:
23. Գրել ծրագիր, որը կանվանափոխի *data.txt* ֆայլի անունը *new_data.txt*-ով:
24. Գրել ծրագիր, որը վերադարձնում է ֆայլի չափը բայթերով:
25. Գրել ծրագիր, որը ստուգում է, եթե ֆայլի չափը մեծ է 5000 բայթից, ջնջում է այն հակառակ դեպքում անվանափոխում է *data.txt*-ից դարձնելով *correct_data.txt* :

Թեմա 3. Գրադարաններ (libs)

NumPy

NumPy-ը (Numerical Python) Python-ի գրադարան է, որը նախատեսված է թվային հաշվարկների համար՝ տրամադրելով բարձր արդյունավետության զանգվածային կառուցվածքներ ու մաթեմատիկական գործիքներ: Այն հատկապես օգտակար է մեծ տվյալների հետ աշխատելիս, երբ անհրաժեշտ է արագություն և մաքսիմալ արդյունավետություն:

Հիմնական կառուցվածքներ.

- NumPy-ի հիմնական կառուցվածքը կոչվում է զանգված (*array*), որը միանման տիպի տարրերի հավաքածու է: Չանգվածները շատ ավելի արդյունավետ են հիշողության և պրոցեսորի առումով, քան Python-ի բազային ցուցակները:
- 1D զանգվածներ: Վեկտորների նման միաչափ զանգվածներ, օրինակ՝
[1, 2, 3, 4],
- 3D զանգվածներ (մատրիցաներ), օրինակ՝ [[1,2], [3, 4]],
- 3D զանգվածներ՝ անսահմանափակ չափերի կառուցվածքներ:

M մատրիցի տարրերին կարելի է հասնել նրանց ինդեքսներով: Ինդեքսավորումը սկսում է 0-ից:

Օրինակ ունենք հետևյալ M մատրիցը:

5	11	15
6	8	10
3	7	9

Առաջին տողի երկրորդ տարրին՝ 11-ին, կարելի է հասնել հետևյալ կերպ M[0,1], կամ երրորդ տողի երկրորդ տարրին՝ 7-ին, կարելի է հասնել հետևյալ կերպ՝ M[2,1]:

Չանգվածի ստեղծում

- *np.array([1, 2, 3])* : Ստեղծում է զանգված նշված արժեքներով:
- *np.zeros((3, 3))* : Ստեղծում է 3x3 զրոյների մատրիցա:
- *np.ones((2, 2))* : Ստեղծում է 2x2 մեկերի մատրիցա:
- *np.arange(0, 10, 2)* : Ստեղծում է 0-ից 10 միջակայքում թվերի հերթականությունը՝ 2 քայլով:
- *np.linspace(0, 1, 5)* : Ստեղծում է 0-ից 1 միջակայքի 5 հավասարապես բաշխված թվեր:
- *np.eye(3)* : Ստեղծում է 3x3 միավոր մատրիցա:

Մաթեմատիկական օպերացիաներ

- *np.add(a, b)* : Գումարում է a-ն և b-ն:
- *np.subtract(a, b)* : Հանում է b-ն a-ից:
- *np.multiply(a, b)* : Տարր առ տարր բազմապատկում է a-ն և b-ն:
- *np.divide(a, b)* : Տարր առ տարր բաժանում է a-ն b-ով:

- `np.sum(A)` : Գումարում է A մատրիցը:

Հատուկ ֆունկցիաներ

- `np.exp(a)` : Հաշվում է e^a :
- `np.sqrt(a)` : Հաշվում է a -ի քառակուսի արմատը:
- `np.log(a)` : Հաշվում է բնական լոգարիթմը:
- `np.sin(a)`, `np.cos(a)` : Հաշվում է a -ի սինուսը և կոսինուսը:

Գծային հանրահաշվային գործողություններ

- `np.dot(a, b)`: Գտնում է մատրիցաների a և b կրկնապատկման արդյունքը:
- `np.linalg.det(a)` : Հաշվում է քառակուսի մատրիցայի որոշիչը
- `np.linalg.inv(a)` : Հաշվում է մատրիցայի հակադարձը (եթե մատրիցան հակադարձելի է)
- `np.linalg.eig(a)` : Հաշվում է քառակուսի մատրիցայի *eigen* արժեքները և *eigen* վեկտորները:
- `np.linalg.solve(a, b)` : Լուծում է գծային հավասարումների համակարգը՝

$$\begin{matrix} \text{Օրինակ՝} & 2x + 3y = 8 \\ & 3x + 4y = 11 \end{matrix}$$

Լուծենք այս համակարգը՝

```
Python
import numpy as np

# Գծային հավասարումների գործակիցների մատրիցա
A = np.array([[2, 3], [3, 4]])

# Սահմանափակումների սյունի
B = np.array([8, 11])

# Լուծում ենք Ax = B համակարգը
solution = np.linalg.solve(A, B)

print("x և y արժեքները՝", solution)
```

Վիճակագրության տարրեր

- `np.mean(a)`: Հաշվում է միջինը:
- `np.median(a)` : Հաշվում է մեդիանը:
- `np.var(a)` : Հաշվում է վարիացիան:
- `np.std(a)` : Հաշվում է ստանդարտ շեղումը:
- `np.sum(a)` : Հաշվում է տարրերի գումարը:
- `np.min(a)`, `np.max(a)` : Գտնում է նվազագույնն ու առավելագույնը:
- `np.corrcoef(x, y)[1, 0]`: Հաշվում է x և y տվյալների [կորրելացիան](#):
- `np.cov(x, y)[1, 0]` : Հաշվում է x և y տվյալների [կովարիացիան](#):

Կորելացիան և կովարացիան հաշվելիս տալիս է 2x2 չափանի մատրից, որտեղ՝

- [0, 0] տարրը **X**-ի փոփոխականի վարիացիան/կորելացիան է
- [1, 1] տարրը **Y**-ի փոփոխականի վարիացիան/ կորելացիան է
- [0, 1] և [1, 0] տարրերը ցույց են տալիս **X** և **Y**-ի միջև կովարիացիան/ կորրելացիան

Տրամաբանական օպերատորներ

- *np.where(condition, x, y)* : Վերադարձնում է *x*-ը, եթե պայմանը բավարարվում է, հակառակ դեպքում՝ *y*:
- *np.any(a)* : Ստուգում է՝ արդյոք զանգվածում կա գոնե մեկ *True* արժեք:
- *np.all(a)* : Ստուգում է՝ արդյոք բոլոր արժեքները *True* են:

Ձևափոխումներ

- *a.reshape((rows, cols))*: Փոփոխում է զանգվածի չափսերը առանց տվյալները փոխելու:
- *a.flatten(a)* : Վերածում է բազմաչափ զանգվածը միաչափի:
- *np.transpose(a)* : Շրջում է մատրիցը:

Տարրերի տեսակավորում և որոնում

- *np.unique(data)* : Վերադարձնում է միայն եզակի տարրերից կազմված ցուցակ:
- *np.sort(a)* : Տեսակավորում է զանգվածը:
- *np.argsort(a)* : Վերադարձնում է տեսակավորման ինդեքսները:
- *np.argmax(a)*, *np.argmin(a)* : Գտնում է առավելագույն և նվազագույն տարրերի ինդեքսները:

Պատահական թվերի գեներացում

- *np.random.random((3, 3))* : Ստեղծում է -1 ,1 միջակայքի պատահական արժեքներով 3x3 չափանի մատրիցա:
- *np.random.randint(- 10, 10, (3, 3))*: Ստեղծում է -10-ից 10 միջակայքում պատահական ամբողջ թվերով 3 x 3 չափանի մատրիցա:
- *np.random.normal(mean, std, size)* : Ստեղծում է նորմալ բաշխված պատահական արժեքների զանգված:

Այս բաժնի խնդիրները լուծելիս պարտադիր է օգտվել *NumPy* գրադարանից:

Մի քանի թվերի ներմուծումները կազմակերպեք ցիկլերի միջոցով, օրինակ *for* ցիկլով:

1. Ներմուծել 5 թիվ, հաշվել այդ թվերի միջինը:

Լուծում՝


```

Python
import numpy as np
L=[]
for i in range (5):
    n = int ( input ("Ներմուծել թիվը"))
    L.append(n)
# Սահմանում ենք սվյալների զանգվածը
data = np.array(L)

# Հաշվում ենք միջինը
mean_value = np.mean(data)
print("Միջինը`", mean_value)

```

- Ստեղծել թվային ցուցակ: Գրել ծրագիր, որը բառարանով վերադարձնում է այդ ցուցակի մեծագույն, փոքրագույն և միջին արժեքները, վարիացիան, ստանդարտ շեղումը և մեդիան:
- Ստեղծել թվային ցուցակ: Գրել ծրագիր, որը տպում է այդ ցուցակի եզակի տարրերից կազմված ցուցակը: Օգտվել `np.unique` ֆունկցիայից:
- Ունենք 10 Նորածին երեխաների քաշերը` [2.9, 3.3, 3.1, 3.6, 2.8, 3.5, 4.0, 3.2, 3.8, 2.7], հաշվել Նորածինների զանգվածի միջինը և ստանդարտ շեղումը:
- Նորածինների քաշերի` [2.9, 3.3, 3.1, 3.6, 2.8, 3.5, 4.0, 3.2, 3.8, 2.7] տվյալների համար հաշվել վարիացիայի գործակիցը $cv = \frac{\text{ստանդարտ շեղում}}{\text{տվյալների միջին}} \times 100$:
- Հողագործները մշակում են այգին: Նրանց աշխատաժամերի և մշակված հողակտորի մակերեսները ներկայացված են հետևյալ աղյուսակով:

X ժամ	1	2	3	4	5
Y հա	3	6	9	12	15

Գտեք **X** և **Y** փոփոխականների միջև կովարիացիան և կորելացիան:

- Ստեղծել [2, 0, 1, 3] զանգվածը և նրա վրա կիրառել `np.sqrt` ֆունկցիան: Կստանանք Նոր զանգված, որի յուրաքանչյուր անդամ նախորդ անդամի զանգվածի քառակուսի արմատն է:
- Ստեղծել [2, 0, 1, 3] զանգվածը և նրա վրա կիրառել `np.exp` ֆունկցիան: Կստանանք Նոր զանգված, որի յուրաքանչյուր անդամ էսպոնենցիալ ֆունկցիայի արժեքն է նախորդից զանգվածի համապատասխան կետում:
- Ստեղծել 4×4 չափանի միավոր մատրից: Տպել այդ մատրիցը:
- Ստեղծել 4×4 չափանի մատրից [0,100] միջակայքի պատահական թվերով: Տպել այդ մատրիցը և նրա որոշիչը:
- Ստեղծել 2×3 և 3×4 չափանի պատահական մատրիցներ 0-ից 10 միջակայքի ամբողջ թվերից կազմված: Տպեք այդ մատրիցները և նրանց արտադրյալը:

12. Ստեղծել 3×3 չափանի երկու պատահական մատրիցներ, տպել այդ մատրիցները և նրանց գումարը:

13. Numpy գրադարանով լուծել հետևյալ գծային հավասարումների համակարգը:

$$3x + 2y = 7$$

$$x - y = 1$$

14. Numpy գրադարանով լուծել հետևյալ գծային հավասարումների համակարգը:

$$x + 2y + 3z = 9$$

$$2x + 3y + z = 82$$

$$2x + y + 2z = 7$$

15. Ստեղծել 3×3 չափանի մատրից $[0,100]$ միջակայքի պատահական թվերով: Տպել այդ մատրիցը և նրա տրանսպոնացված մատրիցը:

16. Ստեղծել 3×3 չափանի մատրից $[0,100]$ միջակայքի պատահական թվերով: Տպել այդ մատրիցի և նրա տրանսպոնացված մատրիցի որոշիչները: Համեմատել այդ որոշիչները:

17. Ներմուծել n թիվ, *numpy* գրադարանով հաշվել և տպել e^n -ը:

18. Ստեղծել 3×3 չափանի մատրից $[0,20]$ միջակայքի պատահական թվերով: Տպել այդ մատրիցի բոլոր տարրերի գումարը:

19. Ստեղծել 4×4 չափանի մատրից $[0,50]$ միջակայքի պատահական թվերով: Հաշվել և տպել բոլոր այն թվերի քանակը, որոնք մեծ են մատրիցի բոլոր թվերի միջինից:

20. Ստեղծել 4×4 չափանի մատրից $[0,50]$ միջակայքի պատահական թվերով: Գլխավոր անկյունագծի տարրերը դարձնել 0 : Տպել ստացված մատրիցը:

Լուծում՝

Python

```
import numpy as np
```

```
# Ստեղծում ենք 4x4 չափի պատահական ամբողջ թվերով մատրից 0-100 միջակայքում  
matrix = np.random.randint(0, 50, (4, 4))
```

```
# Տպում ենք նախնական մատրիցը  
print("Նախնական մատրից:\n", matrix)
```

```
for i in range(4):  
    matrix[i,i] = 0
```

```
# Տպում ենք ստացված մատրիցը  
print("Նոր մատրից", matrix)
```

21. Ստեղծել 4×4 չափի մատրից $[0,50]$ միջակայքի պատահական թվերով:
Գլխավոր անկյունագծից ներքև բոլոր տարրերը դարձնել 1: Տպել ստացված մատրիցը:
22. Ստեղծել 4×4 չափի մատրից $[0,50]$ միջակայքի պատահական թվերով: Հաշվել գլխավոր անկյունագծից վերև գտնվող բոլոր տարրերի գումարը:
23. Ստեղծել 4×4 չափի, պատահական ամբողջ թվերով գեներացվող սիմետրից մատրից:
 M մատրիցը կոչվում է սիմետրից, եթե $M[i, j] = M[j, i]$, բոլոր i, j -երի համար:
24. Ստեղծել 4×4 չափի մատրից $[0,20]$ միջակայքի պատահական թվերով:
Նորմալացնել մատրիցը $\min - \max$ -ի եղանակով և տպել այն:
 $M[i, j] = \frac{A[i, j] - \min(A)}{\max(A) - \min(A)}$, որտեղ $M[i, j]$ -ն նոր մատրիցի համապատասխան տարրն է, $A[i, j]$ -ն սկզբնական մատրիցը, $\min(A)$ -ն և $\max(A)$ -ն համապատասխանաբար A -ի փոքրագույն և մեծագույն արժեքները:
Մատրիցի նորմալացումը դա մաթեմատիկական մի գործողություն է, որի արդյունքում մատրիցայի բոլոր տարրերը համապատասխանեցվում են որոշակի միջակայքի՝ սովորաբար $[0, 1]$:

Matplotlib Pyplot

Matplotlib-ը Python-ի հայտնի գրադարան է, որը թույլ է տալիս ստեղծել գրաֆիկական պատկերներ՝ օգտագործելով պարզ ու հարմար գործիքներ: Այն հարմար է գրաֆիկներ տարբեր տեսակի դիագրամներ կազմելու համար:

Հիմնական հրահանգներ

- `import matplotlib.pyplot as plt` : Ներմուծում է Matplotlib-ի `pyplot` ենթամոդուլը, որը հիմնականում օգտագործում են բոլոր տեսակի գրաֆիկներ ստեղծելու համար:
- `plt.show()` : Ցուցադրում է գրաֆիկը էկրանին, անհրաժեշտ է յուրաքանչյուր գրաֆիկի վերջում, որպեսզի այն տեսանելի լինի:

Գծային գրաֆիկ (Line Plot)

- `plt.plot(x, y)` : Ստեղծում է գծային գրաֆիկ՝ օգտագործելով x և y տվյալները:
- Հատկանիշներ՝ `marker` (կետերի տեսքը), `color` (գույնը), `linestyle` (գծի տեսքը), `ms` (կետի չափը): Ավելի մանրամասն կարող եք տեսնել այստեղ:

Օրինակ՝

```
Python
import matplotlib.pyplot as plt

# Գծային գրաֆիկի օրինակ
```



```
x = [1, 2, 3, 4]
y = [1, 4, 9, 16]
plt.plot(x, y, color='skyblue', linestyle='--', marker='o')
plt.show()
```

Նույն առանցքի վար կարելի է ստեղծել 2 գծային գրաֆիկ`

```
Python
import matplotlib.pyplot as plt

# Տվյալներ
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct",
"Nov", "Dec"]
temperature = [3, 5, 9, 13, 18, 21, 25, 24, 20, 14, 8, 4] # ջերմաստիճան (°C)
precipitation = [78, 54, 62, 48, 71, 67, 58, 76, 82, 98, 89, 77] # տեղումների
մակարդակ (mm)

# Ստեղծում ենք գծապատկեր՝ առաջին y-առանցքով
fig, ax1 = plt.subplots(figsize=(10, 6))

# Չծում ենք ջերմաստիճանը
ax1.plot(months, temperature, color="red", marker="o", label="Temperature
(°C)")
ax1.set_xlabel("Months")
ax1.set_ylabel("Temperature (°C)", color="orange")
ax1.tick_params(axis="y", labelcolor="tab:red")

# Ստեղծում ենք երկրորդ y-առանցքը նույն x-առանցքի համար
ax2 = ax1.twinx()
ax2.plot(months, precipitation, color="darkblue", marker="x",
label="Precipitation (mm)")
ax2.set_ylabel("Precipitation (mm)", color="tab:blue")
ax2.tick_params(axis="y", labelcolor="tab:blue")

# Վերնագիր
plt.title("Temperature and Precipitation by Month")

# Ցույց ենք տալիս գծապատկերը
plt.show()
```

Սյունակաձև դիագրամ (Bar Plot)

- *plt.bar(x, height)* : Ստեղծում է ուղղահայաց սյունակային գրաֆիկ՝ օգտագործելով *x* արժեքները և համապատասխան բարձրությունները:
- *plt.barh(y, width)* : Ստեղծում է հորիզոնական սյունակային գրաֆիկ, օգտագործելով *x* արժեքները և համապատասխան բարձրությունները:
Հատկանիշներ՝ *color* (գույն), *width* (հաստություն):

Օրինակ՝

```
Python
import matplotlib.pyplot as plt

# Տվյալներ
months = ['Հունվար', 'Փետրվար', 'Մարտ', 'Ապրիլ', 'Մայիս']
sales = [1500, 1800, 1200, 1700, 2000]

# Սյունակային գրաֆիկի կառուցում
plt.bar(months, sales, color='#FF7F50', width=0.6)

# Գրաֆիկի ցուցադրում
plt.show()
```

```
Python
import matplotlib.pyplot as plt

# Տվյալներ
months = ['Հունվար', 'Փետրվար', 'Մարտ', 'Ապրիլ', 'Մայիս']
sales = [1500, 1800, 1200, 1700, 2000]

# Սյունակային գրաֆիկի կառուցում
plt.barh(months, sales, color='c', height=0.7)

# Գրաֆիկի ցուցադրում
plt.show()
```

Տարածքային գրաֆիկ (Area Plot)

- *plt.fill_between(x, y1, y2)* : Նկարագրում է երկու գծերի միջև գտնվող տարածքը՝ օգտագործելով *x* արժեքները և *y1*, *y2* սահմանները:
Հատկանիշներ՝ *color* (գույն), *alpha* (թափանցիկություն):

Օրինակներ՝

Python

```
import matplotlib.pyplot as plt

# Տվյալներ
days = [1, 2, 3, 4, 5]
temperature_city1 = [20, 22, 23, 24, 26]
temperature_city2 = [18, 20, 21, 22, 23]

# Տարածքային գրաֆիկի կառուցում
plt.fill_between(days, temperature_city1, temperature_city2, color='darkblue',
alpha=0.4)

# Գրաֆիկի ցուցադրում
plt.show()
```

Python

```
import matplotlib.pyplot as plt

# Տվյալներ
days = [1, 2, 3, 4, 5]
temperature_city1 = [20, 21, 23, 24, 26]
temperature_city2 = [18, 24, 21, 20, 19]

# Տարածքային գրաֆիկի կառուցում
plt.fill_between(days, temperature_city1, color='skyblue', alpha=0.4,
label='Քաղաք 1')
plt.fill_between(days, temperature_city2, color='orange', alpha=0.4,
label='Քաղաք 2')

# Գրաֆիկի ցուցադրում
plt.show()
```

Ֆրված գրաֆիկ (Scatter Plot)

- `plt.scatter(x, y)`: Ստեղծում է ֆրված գրաֆիկ՝ օգտագործելով x և y տվյալները: Հատկանիշներ՝ `color` (գույն), `marker` (կետի տեսք), `s` (կետի չափ):

Օրինակ՝

-

Python

```
import matplotlib.pyplot as plt
```



```
# Տվյալներ
math = [55, 62, 13, 44, 65, 60, 78, 81, 45, 91, 32, 65, 87, 85, 54, 76]
python = [55, 60, 61, 67, 70, 74, 78, 85, 58, 92, 44, 55, 90, 10, 36, 47]

# Ցրված գրաֆիկի կառուցում
plt.scatter(math, python, color='limegreen', marker='o', s=100)

# Գրաֆիկի ցուցադրում
plt.show()
```

Հիստոգրամ (Histogram)

- `plt.hist(data, bins = n)` : Կառուցում է հիստոգրամ:
Հատկանիշներ՝ `color` (գույն), `edgecolor` (եզրագծի գույն), `alpha` (թափանցիկություն):

Օրինակ՝

```
Python
import matplotlib.pyplot as plt

# Տվյալներ (օրինակ՝ 1000 պատահական թիվ)
data = [100, 10, 80, 78, 90, 50, 78, 14, 56, 80, 99, 78, 88]

# Հիստոգրամի կառուցում
plt.hist(data, bins=10, color='gold', edgecolor='red', alpha=0.7)

# Գրաֆիկի ցուցադրում
plt.show()
```

Շրջանային դիագրամ (Pie Chart)

- `plt.pie(sizes, labels = labels)` : Ստեղծում է շրջանաձև դիագրամ:
Հատկություններ՝ `colors` (գույն), `explode` (կտորների հեռավորություն), `autopct` (տոկոսների ցուցադրում), `startangle` (մեկնարկային անկյուն):

Օրինակ՝

```
Python
import matplotlib.pyplot as plt

# Տվյալներ
```

```

chart_labels = ['Արտադրություն', 'Վաճառք', 'Գործառույթ', 'Պահեստավորում']
sizes = [35, 25, 24, 25] # Տոկոսային արժեքներ
chart_colors = ['#FF9999', 'red', '#99FF99', '#FFCC99'] # Գույներ
յուրաքանչյուր հասվածի համար
chart_explode = (0, 0.1, 0.05, 0.08) # Որոշ հասվածների առանձնացում

# Կտրային գրաֆիկի կառուցում
plt.pie(sizes, labels=chart_labels, colors=chart_colors, explode=chart_explode,
autopct='%1.1f%%', startangle=280)

# Գրաֆիկի ցուցադրում
plt.show()

```

Պղպջակային դիագրամ (Bubble Chart)

- `plt.scatter(x, y, s = size)` : Ցրված գրաֆիկի տեսակ է, որտեղ յուրաքանչյուր կետ (բաղադրիչ) ներկայացվում է որպես պղպջակ: Պղպջակի դիրքը (X և Y արժեքները) որոշվում է X , Y տվյալների երկու փոփոխականներով, իսկ պղպջակի չափը՝ երրորդ փոփոխականով:
Հատկություններ՝ *Color* (գույն), *alpha* (թափանցիկություն), *cmap* (գրադիենտային գունավորում, որն ունի '*viridis*', '*plasma*', '*inferno*', '*coolwarm*' ոճերը)

Օրինակ՝

```

Python
import matplotlib.pyplot as plt

# Տվյալներ
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 30, 25]
size = [100, 222, 500, 400, 350] # Պղպջակի չափը
colors = [1, 2, 3, 4, 5] # Պղպջակների գույնը

# Bubble Chart-ի կառուցում՝ գրադիենտով գույնով
plt.scatter(x, y, s=size, c=colors, cmap='plasma', alpha=0.6)

# Վերնագիր և առանցքային պիսակներ
plt.title('Պղպջակային դիագրամ գրադիենտային գունավորմամբ')
plt.xlabel('X արժեքներ')
plt.ylabel('Y արժեքներ')

```

```
# Գրաֆիկի ցուցադրում
plt.colorbar() # Վկելացնել գույնի սանդղակ
plt.show()
```

Տուփային դիագրամ (Box Plot)

• `plt.boxplot(data)` : Ստեղծում է տուփային դիագրամ, ըստ տրված տվյալների Օրինակ՝

```
Python
import matplotlib.pyplot as plt

# Գործիչների քաշերը կգ-երով
newborn_weights = [3.2, 3.5, 2.8, 3.1, 4.0, 2.9, 3.3, 3.6, 2.7, 3.0, 3.4, 3.1,
3.8, 2.6, 3.9, 3.2, 3.5, 3.0, 2.5, 3.7]

# Տուփի դիագրամի կառուցում
plt.figure(figsize=(8, 6)) # չափեր
plt.boxplot(newborn_weights, vert=False, patch_artist=True,
boxprops=dict(facecolor='lightblue'))

# Վկելացնում ենք վերնագիր և պիսակներ
plt.title("Գործիչների քաշերի բաշխում")
plt.xlabel("Քաշը (կգ)")

# Ցույց ենք տալիս դիագրամը
plt.show()
```

Box plot կարելի է նաև կառուցել, եթե ունենք մեկից ավելի տվյալների խմբեր:

```
Python
import matplotlib.pyplot as plt

# Գործիչների քաշերը կգ-երով
newborn_boys = [3.2, 3.5, 2.8, 3.1, 4.0, 2.9, 3.3, 3.6, 2.7, 3.0,]
newborn_girls= [3.4, 3.1, 3.8, 2.6, 3.9, 3.2, 3.5, 3.0, 2.5, 3.7, 2.8]

# Տուփի դիագրամի կառուցում
```

```
plt.figure(figsize=(8, 6)) #չափեր
plt.boxplot([newborn_girls, newborn_boys], labels=['Մեծիկ', 'Տղա' ],
,vert=False, patch_artist=True, boxprops=dict(facecolor='lightblue'))

# Վկելացնում ենք վերնագիր և պիտակներ
plt.title("Նորածինների քաշների բաշխումը")
plt.xlabel("Քաշը (կգ)")
# Ցույց ենք տալիս դիագրամը
plt.show()
```

Տուփ (Box)-ը ցույց է տալիս տվյալների երկրորդ և երրորդ քառորդների (25% – 75%) միջակայքը: Տուփի ներսում գիծ է, որը ցույց է տալիս տվյալների կենտրոնական արժեքը՝ մեդիանը: Տուփից ձգվում են գծեր, որոնք ընդգրկում են հիմնական բաշխումը՝ բաց թողնելով եզրային արժեքները, իսկ եզրային արժեքները տուփի և ուսածողների սահմաններից դուրս գտնվող կետերն են, որոնք տարբերվում են տվյալների ընդհանուր բաշխումից:

Գրաֆիկի ձևավորում

- *plt.title("Գրաֆիկի տիտղոս")* : Սահմանում է գրաֆիկի վերնագիրը:
- *plt.xlabel("X առանցքի անուն")* : Սահմանում է X առանցքի անունը:
- *plt.ylabel("Y առանցքի անուն")* : Սահմանում է Y առանցքի անունը:
- *plt.legend()* : Գրաֆիկին ավելացնում է պիտակ (*label*)՝ յուրաքանչյուր գույնի կամ գծի նշանակությունը ցուցադրելու համար:
- *plt.style.use("style name")* : Այս ֆունկցիան թույլ է տալիս կիրառել նախապես սահմանված ոճեր՝ գրաֆիկների արտաքին տեսքը ձևափոխելու համար: matplotlib-ը պարունակում է մի քանի ներկառուցված ոճեր, ինչպիսիք են՝ *ggplot*, *seaborn*, *bmh*, և այլն:

Օրինակ՝

```
Python
import matplotlib.pyplot as plt

# Ընտրում ենք ոճը , այս ոճը ունի իր գույները և այլ գույներ սահմանել պեժֆ չէ
plt.style.use('tableau-colorblind10')

# Տվյալներ
x = [1, 2, 3, 4, 5]
y1 = [1, 4, 9, 16, 25]
y2 = [1, 3, 6, 10, 15]
```



```

# գծերի գծագրում
plt.plot(x, y1, label="y = x^2")
plt.plot(x, y2, label="y = x * (x + 1) / 2")

# Լեգենդի ցուցադրում
plt.legend()
# Գրաֆիկի ցուցադրում
plt.title("Կոորդինատային հարթությունում y1 և y2 ֆունկցիաները")
plt.xlabel("X արժեքներ")
plt.ylabel("Y արժեքներ")

plt.show()

```

Խնդիրներ

25. Ստեղծել գծային գրաֆիկ՝ $y = 3x$ գծի համար: Գրաֆիկի համար x -ին տալ հետևյալ արժեքները՝ 0, 1, 2, 3:
26. Ստեղծել հետևյալ ֆունկցիայի $y = x^2$ գրաֆիկը, x -ին արժեքները տալ -5 -ից մինչև 5, ընտրելով ամբողջ թվերը:
27. Ստեղծեք գծային գրաֆիկ՝ ներկայացնելու ջերմաստիճանի փոփոխությունը ժամը 8:00-ից մինչև 20:00-ը, ըստ հետևյալ տվյալների.

```

Python
Ժամեր = [8, 10, 12, 14, 16, 18, 20]
Ջերմաստիճան (°C) = [10, 15, 20, 22, 21, 18, 16]

```

28. Կառուցել սյունակաձև դիագրամ՝ ՀՀ ամենաշատ բնակեցված քաղաքների բնակչության տվյալներով՝

```

Python
Քաղաքներ = ['Երևան', 'Գյումրի', 'Վանաձոր', 'Յրազդան', 'Աբովյան', 'Կապան']
Բնակչություն = [1000000, 120000, 80000, 40000, 35000, 30000]

```

29. Կառուցել հիստոգրամ՝ ցույց տալու ուսանողների գնահատականների բաշխումը հետևյալ տվյալներով.

Python

```
Գնահատականներ = [55, 78, 67, 90, 83, 45, 88, 90, 79, 65, 84, 92, 70]
```

30. Կառուցել հիստոգրամ, որը ցույց կտա Ասիա մայրցամաքի տրված լեռների բարձրությունների բաշխումը:

Python

```
Լեռների բարձրություն (մ) = [8848, 8611, 8586, 8516, 8463, 8201, 8167, 8125, 8091, 8078, 8051, 8010, 7925, 7825, 7788, 7690, 7555, 7546, 7526, 7500]
```

31. Հիստոգրամով ներկայացնել տարբեր տարիների վազքի մրցույթների հաղթողների տարիքը:

Python

```
Տարիքներ = [25, 32, 30, 38, 30, 35, 35, 28, 25, 32, 26, 21, 29, 24, 28, 19]
```

32. Կառուցել շրջանային դիագրամ՝ ցուցադրելով արշավական տարբեր պարագաների տեսակների վաճառքի տոկոսային բաժինը:

Python

```
Պարագաներ = ['Կոշիկ', 'Ուսապարկ', 'Թերմոս', 'Քնապարկ', 'Վրան', ]  
Վաճառք (հազ. դրամ) = [400, 200, 30, 240, 270]
```

33. Կառուցել շրջանային դիագրամ՝ ցուցադրելով քննության արդյունքների տոկոսային բաժինը:

Python

```
Պարագաներ = ['Վնբավարար', 'Բավարար', 'Լավ', 'Գերազանց', ]  
Վաճառք (հազ. դրամ) = [25, 12, 30, 17]
```

34. Կառուցել տարածքային գրաֆիկ (Area Plot)՝ ներկայացնելով եկամուտներն ու ծախսերը հետևյալ ամիսների համար.

Python

```
Ամիսներ = ['Հունվար', 'Փետրվար', 'Մարտ', 'Ապրիլ']  
Եկամուսներ ($) = [2000, 2200, 1800, 2100]  
Ծախսեր ($) = [1500, 1600, 1300, 1500]
```

35. Կազմել տարածքային գրաֆիկ (Area Plot), որը ցույց կտա Երևանի, Գյումրիի և Վանաձորի միջին ջերմաստիճանները մի քանի ամիսների ընթացքում, հետևյալ տվյալներով:

Python

```
Ամիսներ = ['Սեպտեմբեր', 'Հոկտեմբեր', 'Նոյեմբեր', 'Դեկտեմբեր', 'Հունվար', 'Փետրվար']  
Երևան=[23, 15, 8, 3, -1, 2]  
Գյումրի=[20, 13, 6, 0, -5, -2]  
Վանաձոր=[22, 14, 7, 2, -3, 0]
```

36. Կազմել ցրված գրաֆիկ (scatter plot)՝ ցուցադրելով մարդկանց հասակի և քաշի տվյալները.

Python

```
Հասակ (սմ) = [160, 165, 170, 175, 180, 185, 163]  
Քաշ (կգ) = [50, 55, 63, 70, 75, 85, 77]
```

37. Կազմել ցրված գրաֆիկ՝ ցուցադրելով աշակերտների գնահատականների տվյալները:

Python

```
Հանրահաշիվ = [25, 78, 45, 90, 95, 55, 16, 67, 24, 100, 32, 65]  
Անգլերեն = [36, 84, 80, 92, 65, 45, 48, 52, 90, 76, 39, 51]
```

38. Կազմել պղպջակային դիագրամ (bubble chart), որտեղ x -ը կլինի ծովի մակարդակից բարձրությունը, y -ը օդի խտությունը, իսկ պղպջակի չափը կբնութագրի արևի վտանգավոր ճառագայթների ինտենսիվությունը:

ԲԾՄ, մ	Օդի խտությունը գ/մ3	Արևի վտանգավոր ճառագայթների ինտենսիվության չափը
--------	---------------------	---

1500	1225	1050
2000	1210	1100
2500	1200	1120
3000	1180	1170
3500	1155	1250
4000	1120	1350

39. Կազմել պոլպչակային դիագրամ, որտեղ x -ը կլինի քաղաքի կանաչ տարածքի մակերեսը, y -ը՝ ամռան ամսիկների միջին ջերմաստիճանը, իսկ պոլպչակի չափը կբոլնթագրի կանաչ տարածքների պահպանման ծախսը:

Կանաչ տարածքի մասնաբաժինը, %	Ամառվա միջին ջերմաստիճան, °C	Ծախս (հազար \$)
25	37	105
45	30	90
60	32	150
75	25	85

40. Ունենք Նեպալի Նամչե Բազար գյուղի բնակիչների հասակների տվյալները:
Կազմել
տոկիային դիագրամ (box plot) նրանց հասակների տվյալներով`

Python

```
height= [150,165, 170, 155, 180, 160, 158, 172, 164, 210, 175, 169, 175, 178, 167, 163, 159, 161, 173,198]
```

Քանի՞ եզրային արժեք կա:

41. Առկա է ուսանողների գնահատականները հետևյալ առարկաներից՝ հանրահաշիվ, անգլերեն և ֆիզիկա: Կազմել տոկիային դիագրամ (box plot) այդ տվյալներով`

Python

```
Հանրահաշիվ = [20, 75, 85, 90, 78, 92, 88, 24, 76, 83, 44, 95, 81,0]
```

```
Վնգլերեն = [70, 80, 65, 88, 55, 91, 84, 72, 86, 70, 100, 79, 92, 30]
```


Ֆիզիկա = [40, 52, 65, 68, 45, 62, 46, 68, 75, 80, 99, 65, 17, 11]

42. Հավաքել տվյալներ ձեր բնակավայրի ամսական միջին տեղումների մասին և կազմել գծային դիագրամ՝ ներկայացնելով տեղումների քանակի փոփոխությունը:
43. Հավաքել տվյալներ համաշխարհային էներգիայի աղբյուրների տեսակների (օրինակ՝ արևային, քամի, ջրային, ածուխ, գազ) մասին և կազմել շրջանային դիագրամ, ներկայացնելով էներգիայի տեսակների հարաբերակցությունը, ըստ օգտագործման ծավալների:
44. Բաց աղբյուրներից հավաքել աշխարհի խոշորագույն սոցիալական մեդիա հարթակների (օրինակ՝ Facebook, Instagram, TikTok, Youtube,) օգտատերերի քանակի մասին տվյալներ և կազմել շրջանային դիագրամ՝ ցույց տալով օգտատերերի տոկոսային բաշխումը:
45. Հավաքել տվյալներ տարբեր երկրների միջին աշխատավարձերի մասին և կազմել հիստոգրամ՝ ներկայացնելով աշխատավարձերի բաշխվածությունը:
46. Հետազոտել 7 խոշորագույն տեխնոլոգիական ընկերությունների (օրինակ՝ Apple, Microsoft, Google, Amazon, Facebook) վերջին տարվա տվյալները՝ հավաքելով նրանց եկամուտները (Revenue), աշխատողների քանակը (Employees) և շուկայական արժեքը (Market Cap): Կազմել պղպջակային դիագրամ, որտեղ՝
- X առանցքը կներկայացնի եկամուտը,
 - Y առանցքը կներկայացնի աշխատողների քանակը,
 - Պղպջակի չափը կներկայացնի շուկայական արժեքը:
47. Հետաքոտեք Երևանի (կամ այլ տարածաշրջանի) օդի մաքրության ինդեքսը և այդ տվյալներով կառուցեք տուփային դիագրամ (box plot) : Տվյալները կարող եք վերցնել օրինակ՝ wagi.info կայքից:
48. Հավաքել տվյալներ մեկ շաբաթվա ընթացքում որևէ բնակավայրի օրվա առավելագույն և նվազագույն ջերմաստիճանների մասին: Ներկայացնել այն տարածքային գրաֆիկի միջոցով:

Թեմա 4. Խորացված ֆունկցիաներ (Advanced functions)

Լամբդա (Lambda) տեսակի ֆունկցիա

1. Ստեղծել *lambda* ֆունկցիա, որը ստանում է երկու թիվ և վերադարձնում է նրանց գումարը:

2. Ստեղծել *lambda* ֆունկցիա, որը ստանում է թիվ և վերադարձնում է նրա քառակուսին:
3. Ստեղծել *lambda* ֆունկցիա, որը ստանում է *a* , *b* թվերը և վերադարձնում է $2a^3 + b^2$ արտահայտության արժեքը:
4. Ստեղծել *lambda* ֆունկցիա, որը ստանում է *a*, *b*, *c* թվերը և վերադարձնում է $2a^2 - 3ab - 4c^3$ արտահայտության արժեքը:
5. Ստեղծել *lambda* ֆունկցիա, որը ստանում է թիվ և ստուգում է արդյոք այն զույգ է, թե ոչ: (վերադարձնում է *true* կամ *false*)
6. Ստեղծել *lambda* ֆունկցիա, որտ ստանում է տող և տողի բոլոր տառերը դարձնում է մեծատառ:
7. Ստեղծել *lambda* ֆունկցիա, որը ստանում է երկու բառ և վերադարձնում է նրանցից ավելի երկարը:
8. Ստեղծել *lambda* ֆունկցիա, որը ստանում է ջերմաստիճանը ցելսիուսով և վերադարձնում է ջերմաստիճանը ֆարենհայտով:
9. Ստեղծել *lambda* ֆունկցիա, որը ստանում է թիվ և թվին աջից կգագրում է 5:
10. Ստեղծել *lambda* ֆունկցիա, որը ստանում է թիվ և թվին ձախից կցագրում է 5:
11. Ստեղծել *lambda* ֆունկցիա, որը ստանում է թիվ և վերադարձնում է 50, եթե թիվը փոքր է 50-ից, հակառակ դեպքում այդ թվի կրկնապատիկը:
12. Ստեղծել *lambda* ֆունկցիա, որը ստանում է ցուցակ և վերադարձնում է առաջին և վերջին տարրերի կիսագումարը:

Python-ում *filter()* ֆունկցիան օգտագործվում է տարրերը ֆիլտրելու կամ զտելու համար, այն ընդունում է երկու հիմնական արգումենտ՝ ֆունկցիա և iterable օբյեկտ (օրինակ՝ ցուցակ, տող և այլն), ապա վերադարձնում է այն տարրերը, որոնք բավարարում են ֆիլտրի պայմաններին: Օրինակ ցուցակից ֆիլտրենք դրական թվերը՝

```
Python
numbers = [-5, -3, 0, 2, 4, 6] # ճախնական ցուցակ

# lambda ֆունկցիա, որը վերադարձնում է True, եթե թիվը դրական է,
positive = lambda x: x>0

# Ֆիլտրի ֆունկցիա, որը վերադարձնում է դրական թվերը
positive_numbers = list( filter(positive, numbers))

# Տպում ենք դրական թվերը
print((positive_numbers)) # Output: [2, 4, 6]
```

13-ից 25 առաջադրանքները կատարելու համար անհրաժեշտ է *filter*-ի միջոցով ստեղծել նոր ցուցակ (կամ տող) ըստ առաջադրանքի պահանջի և տպել այն:

13. Ցուցակից ֆիլտրել բոլոր զույգ թվերը:

14. Ցուցակից ֆիլտրել 100-ից փոքր թվերը
15. Ցուցակից ֆիլտրել 25-ից մեծ և 75-ից փոքր թվերը:
16. Ցուցակից ֆիլտրել այն տողերը, որոնք սկսում են մեծատառով:
17. Ցուցակից ֆիլտրել այն տողերը, որոնց երկարությունը մեծ է 5-ից:
18. Ցուցակից ֆիլտրել այն տողերը, որոնք պարունակում են "a" տառը:
19. Ցուցակից ֆիլտրել 3-ի բազմապատիկ թվերը:
20. Ցուցակից ֆիլտրել այն տողերը, որոնք ավարտվում են " ! " սիմվոլով:
21. Ցուցակից ֆիլտրել այն տողերը, որոնց երկարությունը կենտ թիվ է:
22. Ստեղծել *filter*, որը պահում է տողի մեջ ամկա միայն տառերը:
23. Ստեղծել *filter*, որը պահում է տողի մեջ ամկա մեծատառերը:
24. Ստեղծել *filter*, որը պահում է տողի մեջ ամկա թվերը:
25. Ստեղծել *filter*, որը պահում է տողի մեջ ամկա ձայնավորները:

map () տեսակի ֆունկցիա

26. Ստեղծել *map()* ֆունկցիա, որը ցուցակի յուրաքանչյուր տարր կրկնապատկում է:
27. Ստեղծել *map()* ֆունկցիա, որը ցուցակի յուրաքանչյուր տարր բարձրացնում է 20%-ով:
28. Ստեղծել *map()* ֆունկցիա, որը ցուցակի յուրաքանչյուր տարր նվազեցնում է 7-ով:
29. Ստեղծել *map()* ֆունկցիա, որը ցուցակի յուրաքանչյուր տարրից վերցնում է նրա 34 մասը:
30. Ստեղծել *map()* ֆունկցիա, որը ցուցակի յուրաքանչյուր թիվ բարձրացնում է խորանարդ աստիճան:
31. Ստեղծել *map()* ֆունկցիա, որը ցուցակի յուրաքանչյուր տողից վերցնում է միայն առաջին տարրը:
32. Ստեղծել *map()* ֆունկցիա, որը գումարում է երկու ցուցակների համապատասխան տարրերը:
33. Ստեղծել *map()* ֆունկցիա, որը ցուցակի յուրաքանչյուր գույգ թիվ բազմապատկում է 2-ով, իսկ կենտ թվերը 3-ով:
34. Ստեղծել *map()* ֆունկցիա, որը ցուցակի յուրաքանչյուր բացասական թվից վերցնում է նրա բացարձակ արժեքը:
35. Ստեղծել *map()* ֆունկցիա, որը ցուցակի յուրաքանչյուր տող շրջում է `hello` → `olleh`:
36. Ստեղծել *map()* ֆունկցիա, որը ստուգում է, արդյոք ցուցակի տողը կազմված է միայն թվերից թե ոչ:

Օրինակ՝

Python

```

Unisf
[ "Hello" , "125" , "ASA15" ]

```

Ելք

```

[ False , True , False ]

```


37. Ստեղծել `map()` ֆունկցիա, որը ստուգում է, արդյոք ցուցակի տարրը ամբողջ թիվ է թե ոչ: Օգտվել `isinstance(x, int)` ֆունկցիայից, որը ստուգում է արդյոք `x` փոփոխականը նշված տիպից է, թե ոչ:
38. Ստեղծել `map()` ֆունկցիա, որը ցուցակի յուրաքանչյուր անուն ազգանուն կրճատում է թողնելով անվան սկզբնատառը և ազգանունը:

Օրինակ՝

Python

Մուտք

Ելք

```
["Anna Karenina", "John Smith", "Peter Johnson"] ['A. Karenina', 'J. Smith', 'P. Johnson']
```

39. Ստեղծել `map()` ֆունկցիա, որը ցուցակի յուրաքանչյուր տողի սկզբում ավելացնում է "Հարգելի" բառը:

Python

Մուտք

Ելք

```
["ՂԱՃԱ", "Գոռ", "Հայկ"] ['Հարգելի՛ ՂԱՃԱ', 'Հարգելի՛ Գոռ', 'Հարգելի՛ Հայկ']
```

40. Ստեղծել `map()` ֆունկցիա, որը ցուցակի յուրաքանչյուր բառի կողքին կգրի հետևյալ դասակարգումը՝ «Կարճ» (մինչև 5 տառ), «Միջին», (5-7 տառ), «Երկար» (7-ից ավելի տառեր):

Լուծում՝

Python

```
words = ["Debed", "Aghstev", "Azat", "Gavarraget", "Araks"]
```

```
classified = list(map(lambda word: (word,
    "Կարճ" if len(word) <= 4 else
    "Միջին" if 5 <= len(word) <= 7 else
    "Երկար"), words))
```

```
print(classified)
```

```
# Արդյունք: [('Debed', 'Միջին'), ('Aghstev', 'Երկար'), ('Aza', 'Կարճ'),
('Gavarraget', 'Երկար'), ('Araks', 'Միջին')]
```

41. Ստեղծել `map()` ֆունկցիա, որը ցուցակի յուրաքանչյուր գնահատականի կողքին կգրի հետևյալ դասակարգումը՝ «Անբավարար» (0 - 40 միավոր) , «Բավարար» (41 - 60 միավոր), «Լավ» (61 - 80 միավոր), «Գերազանց» (81 - 100 միավոր):

`reduce()` տեսակի ֆունկցիա

`reduce()`-ը `functools` մոդուլի օպերատոր է, որը թույլ է տալիս կրկնվող կերպով կիրառել գործողություն (ֆունկցիա) տվյալների հավաքածուի (օրինակ՝ ցուցակի) վրա, մինչև հավաքածուի ավարտը:

Շարահյուսություն (*syntax*)՝

```
Python
from functools import reduce

result = reduce(function, iterable, initializer)
```

- **function**՝ ֆունկցիան է, որը պետք է կիրառվի տարրերի վրա:
- **iterable**՝ տվյալների հավաքածուն է (օրինակ՝ ցուցակ կամ tuple):
- **initializer** (կիրառությունը ըստ ցանկության)՝ լռելայն արժեքն է:

Ինչպե՞ս է աշխատում `reduce()`-ը: Այն քայլ առ քայլ գործարկում է ֆունկցիան հետևյալ ձևով.

- a. Կիրառում է ֆունկցիան հավաքածուի առաջին երկու տարրերի վրա:
- b. Ստացված արդյունքը օգտագործում է որպես առաջին օպերանդ, իսկ հաջորդ տարրը՝ երկրորդ:
- c. Կրկնում է գործընթացը մինչև վերջ:

Օրինակ 1. Հաշվել ցուցակի բոլոր թվերի գումարը:

```
Python
from functools import reduce

numbers = [1, 2, 3, 4, 5]
result = reduce(lambda x, y: x + y, numbers)
print(result) # Արդյունք: 15
```

Օրինակ 2. Գտնել ցուցակի ամենամեծ թիվը:

```
Python
from functools import reduce

numbers = [3, 1, 7, 5, 9, 2]
max_number = reduce(lambda x, y: x if x > y else y, numbers)
print(max_number) # Արդյունք: 9
```

42. Ստեղծեք `reduce()` ֆունկցիա, որը կհաշվի ցուցակի բոլոր թվերի արտադրյալը:

43. Ստեղծեք `reduce()` ֆունկցիա, որ կմիացնի ցուցակի բոլոր տողերը իրար դարձնելով մեկ տող: Տողերը իրարից անջատված լինեն բացառով:

Օրինակ՝

Python

Unisf

```
[ "Ararat" , " Vedi" , " Vayq" , "Kapan" ]
```

Ելք

```
"Ararat Vedi Vay Kapan"
```

44. Ստեղծեք `reduce()` ֆունկցիա, որը կհաշվի բոլոր թվեր բացարձակ արժեքների գումարը:

45. Ստեղծեք `reduce()` ֆունկցիա, որը կգտնի ցուցակի ամենափոքր տարրը:

46. Ստեղծեք `reduce()` ֆունկցիա, որը կգտնի ցուցակի ամենաերկար տողը:

47. Ստեղծեք `reduce()` ֆունկցիա, որը կհաշվի ցուցակի բոլոր թվերի կրկնապատիկների գումարը:

Լուծում՝

Python

```
from functools import reduce
```

```
# Մուտքային ցուցակը
```

```
nums = [1, 2, 3, 4]
```

```
# reduce()-ով լուծումը
```

```
result = reduce(lambda a, x: a + 2 * x, nums, 0)
```

```
print("Կրկնակիների գումարը:", result)
```

48. Ստեղծեք `reduce()` ֆունկցիա, որ կհաշվի ցուցակի բոլոր տարրերի եռապատիկների գումարը:

49. Ստեղծեք `reduce()` ֆունկցիա, որ կհաշվի ցուցակի բոլոր տարրերի 5-ի վրա բաժանելիս ստացված մնացորդների գումարը:

50. Ստեղծեք `reduce()` ֆունկցիա, որ կհաշվի ցուցակի բոլոր տարրերի քառակուսիների գումարը:

51. Ստեղծեք `reduce()` ֆունկցիա, որ կհաշվի ցուցակի բոլոր տողերի երկարությունների գումարը:

52. Ստեղծեք `reduce()` ֆունկցիա, որ հերթականությամբ ցուցակի մի տարր գուլարում է հաջորդը հանում է:

Օրինակ՝

```
Python
#Մոտեմ
[ 1, 5, 7, 12, -2]

#Ելք
14                #1 + 5 - 7 + 12 - (-2) = 13
```

any և all տեսակի ֆունկցիաներ

any() ֆունկցիան բուլյան ֆունկցիա է, որը ստուգում է *iterable*-ի (օրինակ՝ ցուցակի, *tuple*-ի, *set*-ի և այլն) տարրերը և վերադարձնում է *True*, եթե դրանցից գոնե մեկը ճշմարիտ է: Եթե բոլոր տարրերը կեղծ են (*False*), ապա վերադարձնում է *False*

all() ֆունկցիան ստուգում է տրված *iterable*-ի (*list*, *tuple*, *set* և այլն) բոլոր տարրերը և վերադարձնում է *True*, եթե դրանցից բոլորը ճշմարիտ են: Եթե գոնե մեկը կեղծ է (*False*), այն վերադարձնում է *False*

Ճարահյուսություն (syntax)

```
Python
any (iterable)
all (iterable)
```

Օրինակ 1. Ստուգել, թե արդյոք թվային ցուցակում առկա է 0-ից տարբեր թիվ:

```
Python
nums = [0, 0, 5, 0]
print(any(nums))  # True (քանի որ ունենց 5 արժեքը, որը True է)
```

Օրինակ 2. Ստուգել արդյոք թվային ցուցակի բոլոր տարրերը 0-ից տարբեր են թե ոչ:

```
Python
nums = [24, -7, 5, 0]
print(all(nums))  # False (քանի որ ունենց 0 արժեքը, որը False է)
```

53. *any* ֆունկցիայով ստուգել, արդյոք ցուցակում կա ոչ դատարկ տող:

54. *all* ֆունկցիայով ստուգել թվային ցուցակում կա արդյոք 0 արժեք:

55. *any* ֆունկցիայով ստուգել արդյոք ցուցակում կա 7 թիվը:
Լուծում՝

Python

```
nums = [1, 2, 3, 7]
result_list = [num == 7 for num in nums] #սահման եմք այսպիսի ցուցակ [False,
False, False, True]
print(any(result_list)) # True
```

56. *all* ֆունկցիայով ստուգել արդյոք ցուցակի բոլոր տարրերը զույգ են:

57. *any* ֆունկցիայով ստուգել թվային ցուցակում կա արդյոք դրական թիվ:

58. *any* ֆունկցիայով ստուգել արդյոք ցուցակում կա “Gen AI” տողը:

59. *all* ֆունկցիայով ստուգել արդյոք ցուցակի բոլոր տողերը սկսում են “A” տառով (
միայն մեծատառ A-ն դիտարկել):

60. *any* ֆունկցիայով ստուգել արդյոք թվային ցուցակում կա տարր, որը մեծ է իր
նախորդից:

61. *all* ֆունկցիայով ստուգել արդյոք ցուցակի բոլոր տողերի երկարությունը մեծ է
3-ից:

Թեմա 4. Ռեկուրսիա (Recursion)

1. Գրել ռեկուրսիվ ֆունկցիա, որը ստանում է n բնական թիվ և վերադարձնում է 1-ից n թվերի գումարը:
2. Գրել ռեկուրսիվ ֆունկցիա, որը ստանում է n բնական թիվ և տպում է 1-ից n բոլոր
կենտ թվերը:
3. Գրել ռեկուրսիվ ֆունկցիա, որը ստանում է n թիվ և վերադարձնում է 1-ից n զույգ
թվերի գումարը:
4. Գրել ռեկուրսիվ ֆունկցիա, որը ստանում է n թիվ և վերադարձնում է այդ թվի
ֆակտորիալը:
5. Գրել ռեկուրսիվ ֆունկցիա, որը ստանում է n թիվ և վերադարձնում է Ֆիբոնաչչի n
-րդ թիվը:
6. Գրել ռեկուրսիվ ֆունկցիա, որը ստանում է n թիվ և վերադարձնում է Ֆիբոնաչչի
առաջին n թվերի գումարը:
7. Գրել ռեկուրսիվ ֆունկցիա, որը ստանում է n թիվ և վերադարձնում է այդ թվի
թվանշանների գումարը:
8. Գրել ռեկուրսիվ ֆունկցիա, որը ստանում է n և x բնական թվեր և վերադարձնում է
 x^n :
9. Գրել ռեկուրսիվ ֆունկցիա, որը ստանում է a և b բնական թվեր և վերադարձնում է
 $a*b$ արտահայտության արժեքը, առանց բազմապատկման գործողության
օգտագործման:

10. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է a և b բնական թվեր և վերադարձնում է այդ թվերի ամենամեծ ընդհանուր բաժանարարը Էվկլիդեսի մեթոդով:
11. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է n թիվ և վերադարձնում է n թվի համար հարմոնիկ ֆունկցիայի արժեքը: $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$:
12. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է n թիվ և վերադարձնում է n թվի համար այս ֆունկցիայի արժեքը՝ $f(n) = n + 2n + 3n + \dots + nn$:
13. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է n և r թվեր և վերադարձնում է երկրաչափական շարքի գումարը այդ թվերի համար

$$S(n) = 1 + r + r^2 + r^3 + \dots + r^n$$
:
14. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է a և b բնական թվեր ($a < b$) և տպում է a -ից b միջակայքի բոլոր բնական թվերը:
15. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է n թիվ և ստուգում է այն պարզ է թե ոչ:

Լուծում

Python

```
def is_prime(n, divisor=2):
    if n < 2: # 0 և 1 պարզ թվեր չեն
        return False
    if divisor == n: # Եթե բաժանարարը հասել է n-ին անցել է n-ը, n պարզ է
        return True
    if n % divisor == 0: # Եթե n բաժանվում է divisor-ի
        return False
    return is_prime(n, divisor + 1) # Ձեկուրսիվ ստուգում հաջորդ բաժանարարի համար
```

16. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է n թիվ և վերադարձնում է նրա տեսքը երկուական համակարգում:
17. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է տող և վերադարձնում է նրա երկարությունը:
18. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է տող և ստուգում է արդյոք տողը պոլիդրոմ է թե ոչ:
19. Գրել 2 ձեկուրսիվ ֆունկցիաներ, որոնցից առաջինը ստանում է n բնական թիվ և վերադարձնում է n թվի ֆակտորյալը: Երկրորդը ստանում է n բնական թիվ, օգտվելով ֆակտորյալ հաշվող ֆունկցիայից այդ թվի համար հաշվում է e թվի n -րդ շարքի գումարը՝
$$e = 2 + \frac{1}{2!} + \frac{1}{3!} + \dots + \frac{1}{n!} + \dots$$
20. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է թվային ցուցակ և վերադարձնում է բոլոր տարրերի գումարը:
21. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է թվային ցուցակ և վերադարձնում է բոլոր տարրերի արտադրյալը:
22. Գրել ձեկուրսիվ ֆունկցիա, որը ստանում է n բնական թիվ և վերադարձնում է ցուցակով այդ թվի վերլուծությունը պարզ արտադրիչների:

Python

```
def prime_factors(n, divisor=2):  
    pass  
  
number = 50  
print ( prime_factors(number)) # [ 2,5,5]
```

Բաժանարարը սկսում ենք 2-ից, եթե թիվը բաժանվում է divisor-ի վրա, ապա divisor-ը ավելացնում ենք ցուցակում, իսկ ֆունկցիան կիրառում ենք բանորդի և նույն divisor-ի համար, եթե թիվը չի բաժանվում divisor-ի, ապա ֆունկցիան կիրառում ենք նույն թվի և divisor+1-ի համար:

23. Գրել ռեկուրսիվ ֆունկցիա, որը ստանում է թվային ցուցակ և դասակարգում է այն Թոնի Ջորի դասակարգման ալգորիթմով:
Այս ալգորիթմը ընտրում է ցուցակից որևէ տարր և ցուցակը բաժանում է 3 մասի՝ «այդ թվից մեծ թվեր», «այդ թվից փոքր թվեր», «այդ թվին հավասար թվեր»: Հետո ռեկուրսիվ կերպով նույն գործողությունը անուն է այդ երեք մասերի հետ և վերջում միացնում է ստացված մասերը, ինչից հետո ցուցակը դասակարգվում է :

Լուծում՝

Python

```
def quicksort(arr):  
    # Բազային դեպք  
    if len(arr) <= 1:  
        return arr  
  
    # Միջանկյալ տարրի ընտրություն (կարող է լինել ցանկացած տարր)  
    barrier = arr[len(arr) // 2]  
  
    # Ենթաբաժանումներ  
    left = [x for x in arr if x < barrier] # barrier-ից փոքր տարրեր  
    middle = [x for x in arr if x == barrier] # barrier-ին հավասար տարրեր  
    right = [x for x in arr if x > barrier] # barrier-ից մեծ տարրեր  
  
    # Ռեկուրսիա ենթաբաժանումների վրա  
    return quicksort(left) + middle + quicksort(right)  
  
# Օրինակ օգտագործում  
arr = [3, 6, 8, 10, 1, 2, 1]  
sorted_arr = quicksort(arr)  
print(sorted_arr) # [1, 1, 2, 3, 6, 8, 10]
```

Թոնի Ջորի ալգորիթմով ցուցակը դասակարգել նվազման կարգով:

24. Թոնի Զորի դասակարգման ալգորիթմով դասակարգել տողերից կազմված ցուցակը ըստ տողերի երկարությունների:

Օգտագործված գրականություն և արտաքին հղումներ

Python 3 documentation: docs.python.org/3/

GeeksforGeeksonline platform <https://www.geeksforgeeks.org/>

W3 school python tutorial: [w3schools.com/python/default.asp](https://www.w3schools.com/python/default.asp)

Python in Education: education.python.org/resources/resource/list

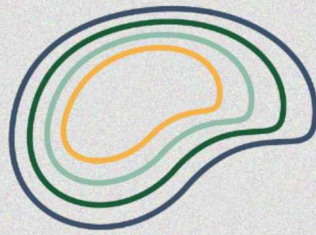
Real Python Tutorials: realpython.com

Python documentation - <https://docs.python.org>

Research is Fun: research1.fun/category/python

PYnative Exercises: pynative.com/python-exercises-with-solutions

NumPy documentation:- <https://numpy.org>



Generation AI

Nurturing Future Innovators



ՀՀ ԿՐԹՈՒԹՅԱՆ, ԳԻՏՈՒԹՅԱՆ,
ՄՇԱԿՈՒՅԹԻ և ՄԴՈՐՏԻ ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ